

AGV 二次开发接口

本文档用于第三方开发者对接 AGV 的串口通信。接口仅涉及 3 个业务结构体与 `uart_frame` 打包/解析 API:

- 业务结构体: `ctrl_frame_t`、`sensor_data_t`、`status_feedback_t`
- 帧层 API: `uart_frame_pack`、`uart_frame_parse`、`uart_frame_find`、`uart_frame_validate_format`

1. 概述与快速开始

- 适用范围: 通过 **UART** 与 AGM MCU 通讯, 实现控制 (**CONTROL_CMD**)、状态查询与反馈 (**STATUS_FEEDBACK**)、以及传感数据接收 (**SENSOR_DATA**)。
- 串口物理参数: **460800 8N1**。
- 字节序与对齐: 小端序; 所有结构体均 `__attribute__((packed))`, 跨平台按字节对齐。

集成步骤 (建议做法, 常见构建系统写法):

- 将以下两个目录加入「头文件搜索路径」 (Header Search Path / Include Directories):
 - `uart_frame/`
 - `agv_uart_frame/`
- 同时把 `uart_frame/` 下的源文件加入工程构建: `uart_frame.c`、`uart_frame_crc.c`。
- 使用时仅需在代码里包含: `#include "uart_frame.h"` (其中会透传包含配置与数据结构定义)。

示例:

- CMake:

```
1 target_include_directories(your_target PRIVATE /path/to/uart_frame
  /path/to/agv_uart_frame)
2 target_sources(your_target PRIVATE /path/to/uart_frame/uart_frame.c
  /path/to/uart_frame/uart_frame_crc.c)
```

2. 术语与统一约定

- 车轮角度: 顺时针旋转, 车轮角度变化范围为 $[90^{\circ}, -90^{\circ}]$, 其中 0° 为车头正前方。
- 角度范围: 左右轮 $\pm 90^{\circ}$; 相机云台 $\pm 30^{\circ}$ (物理活动范围更小)。
- PWM (占空/方向): `pwm_duty_t` 有效范围 $[-100, 100]$; 绝对值越大速度越大; 正负表示方向。
- 编码器脉冲: 车轮每转一圈为 **186** 个脉冲。
- UWB 坐标: `x/y` 单位为 cm, 直角坐标系; `rate` 为 Hz (仅表示发送频率, 可忽略)。
- IMU: `IMU_ARRAY_NUM=5` 表示 100ms 内 5 次采样 (可先取平均再计算); 6DoF/9DoF 欧拉角单位为“度”。
- **SENSOR_DATA** 推送: AGM 以 10Hz 主动推送; 不需请求。
- **STATUS_FEEDBACK** 查询: 请求-响应型; AGM 忽略请求体并返回完整状态 (建议请求体为全零结构体)。

3. 帧协议总览

- 帧头魔术字: `0xAA 0x55`
- 帧尾魔术字: `0x0D 0x0A`
- 协议版本: `0x01`
- 最大数据区: `UART_FRAME_DATA_MAX_SIZE = 1024`
- CRC: CRC16-CCITT, 多项式 `0x1021`, 初始值 `0xFFFF`

帧层 API (简述):

- `uart_frame_pack(type, data, data_len, frame_buf, frame_buf_size, out_frame_len)`: 打包数据为帧
- `uart_frame_parse(frame_buf, frame_len, result)`: 解析帧并返回结果 (含类型与数据指针)
- `uart_frame_find(rx_buf, rx_len, frame_start, frame_length)`: 从接收缓存中定位一帧
- `uart_frame_validate_format(frame_buf, frame_len)`: 快速校验帧头/长度/尾格式

涉及的类型 ID:

- `UART_FRAME_TYPE_SENSOR_DATA = 0x10`
- `UART_FRAME_TYPE_CONTROL_CMD = 0x11`
- `UART_FRAME_TYPE_STATUS_FEEDBACK = 0x14`

4. 数据结构详解

4.1 控制命令 `ctrl_frame_t` (第三方 → AGM)

作用: 用于控制底盘与云台、设置电机占空与运动目标脉冲数, 以及控制指示灯模式。

```
1 // UART_FRAME_TYPE_CONTROL_CMD
2 typedef struct {
3     struct {
4         float_t angle_left; // 左轮转角 (度), 顺时针为正, 范围约 ±90°
5         float_t angle_right; // 右轮转角 (度), 顺时针为正, 范围约 ±90°
6         float_t angle_camera; // 相机云台角度 (度), 顺时针为正, 范围约 ±30°
7     } angles; // 目标角度
8
9     struct {
10         uint32_t chassis_lock : 1; // 车体上锁: 0=解锁, 1=上锁
11         uint32_t motor0_break : 1; // 左电机刹车: 0=关, 1=开
12         uint32_t motor1_break : 1; // 右电机刹车: 0=关, 1=开
13         uint32_t diff_locked : 1; // 差速锁定: 0=关, 1=开
14 #ifdef UART_PID_DEBUG
15         uint32_t pid_valid : 1; // PID 参数有效: 0=无效, 1=有效 (第三方可忽略)
16 #endif
17     } flags;
18
19     struct {
20         int8_t left; // 左轮 PWM, 占空/方向: [-100,100]
21         int8_t right; // 右轮 PWM, 占空/方向: [-100,100]
22         uint16_t reserved; // 保留
```

```

23     uint32_t pulses;    // 本次目标脉冲数（编码器计数）
24 #ifdef UART_PID_DEBUG
25     float kp, ki, kd;  // 调试用，可忽略
26 #endif
27     } motor_rpm;        // 运动控制
28
29     struct {
30         uint32_t red_mode      : 2; // 红灯：00=OFF, 01=ON, 10=BLINK
31         uint32_t blue_mode     : 2; // 蓝灯：00=OFF, 01=ON, 10=BLINK
32         uint32_t defenses_mode : 2; // 防御灯：00=OFF, 01=ON, 10=BLINK
33         uint32_t flow_enable   : 1; // 呼吸/流水：0=关, 1=开
34         uint32_t period_ms     : 12; // 闪烁周期(ms), 0-4095
35         uint32_t duty_cycle    : 7;  // 占空比(0-100)
36         uint32_t reserved      : 6;  // 保留
37     } led_ctrl;          // 灯控
38 } __attribute__((packed)) ctrl_frame_t;

```

关键字段说明：

- `flags`:
 - `chassis_lock` 车体锁：1=上锁（禁止运动）；0=解锁（允许运动）。
 - `motor0_break` 左电机刹车；`motor1_break` 右电机刹车：1=刹车，0=释放。
 - `diff_locked` 差速锁：1=锁定，0=关闭。当开启差速锁时，小车在运动过程中，会自行调节转速，以保证两侧车轮转速一致。
 - `pid_valid`（可选调试）：若开启 `UART_PID_DEBUG`，则可自行定义 `pid` 参数，覆盖小车的默认 `pid` 参数。
- `motor_rpm.left/right`：电机 PWM 占空/方向，取值 `[-100,100]`；绝对值越大速度越大；正负为方向。
- `motor_rpm.pulses`：本次运动目标脉冲数（编码器计数）。以 186 脉冲≈1 圈；常用于设定位移/行程目标。当 `pulses = 0` 时，则会一直运动。
- `led_ctrl`:
 - `*_mode`：00=OFF, 01=ON, 10=BLINK。
 - `flow_enable`：呼吸/流水效果开关。
 - `period_ms`：闪烁周期（ms），0 表示不启用闪烁。
 - `duty_cycle`：占空比（0-100）。

4.2 传感器数据 `sensor_data_t` (AGM → 第三方, 10Hz)

作用：周期性推送 UWB、IMU、超声等传感数据，便于融合与监测。

```

1 // UART_FRAME_TYPE_SENSOR_DATA
2 typedef struct {
3     uint32_t frame_id;    // 传感数据包号
4     uint32_t valid_flags; // 各源数据有效性按位标志
5
6     // UWB 定位（单位：cm）
7     struct {
8         uint32_t packid; // 包 ID
9         int32_t mode;    // 模式

```

```

10     int32_t x;        // X 坐标 (cm)
11     int32_t y;        // Y 坐标 (cm)
12     int32_t rate;     // 发送频率 (Hz, 可忽略)
13 } uwb;
14
15 // IMU (100ms 内 5 次采样; 欧拉角单位: 度)
16 struct {
17     int16_t gyro_raw[3]; // 原始陀螺 [X,Y,Z]
18     int16_t accel_raw[3]; // 原始加计 [X,Y,Z]
19     int16_t mag_raw[3];   // 原始磁力 [X,Y,Z]
20     int16_t reserved;     // 保留 (字节对齐)
21     float temperature;    // 温度 (°C)
22     float accel_x, accel_y, accel_z; // 加速度 (m/s^2)
23     float roll_6dof, pitch_6dof, yaw_6dof; // 6DoF 欧拉角 (度)
24     float roll_9dof, pitch_9dof, yaw_9dof; // 9DoF 欧拉角 (度)
25 } imu[5];
26
27 // 超声 (单位: mm)
28 struct {
29     uint16_t distance1_mm;
30     uint16_t distance2_mm;
31 } uds;
32 } __attribute__((packed)) sensor_data_t;

```

关键字段说明:

- `frame_id`: 数据帧编号 (单调递增), 可用于丢包检测与时序分析。
- `valid_flags`: 有效性标志 (1=无效, 0=有效) 。
 - `DATA_INVALID_UWB / IMU / UDS1 / UDS2`: 对应源数据无效时置位; 建议先检查再使用对应数据。
- `uwb.{x,y}`: 单位 cm 的直角坐标; `rate` 为 Hz, 仅指示发送频率, 可忽略。
- `imu[IMU_ARRAY_NUM]`: 100ms 时间窗内 5 次采样。
 - `*_raw`: 原始三轴数据; `temperature`: 温度 (°C) 。
 - `accel_x/y/z`: m/s^2; `roll/pitch/yaw_6dof/9dof`: 欧拉角 (度) 。
 - 建议对 5 组数据先做简单平均或滤波再计算姿态/速度等衍生量。
- `uds.distance1_mm/distance2_mm`: 超声测距, 单位 mm。

使用建议:

- 先判定 `valid_flags`, 再读取对应源数据; 对于无效源请忽略或维持上次有效值。
- 不依赖浮点跨平台一致性进行协议对接; 跨语言建议优先使用整数字段进行校验。

4.3 状态反馈 `status_feedback_t` (查询-响应)

作用: 第三方按需查询当前执行状态; AGM 返回完整状态。

```

1 // UART_FRAME_TYPE_STATUS_FEEDBACK
2 typedef struct {
3     struct {
4         float_t angle_left;
5         float_t angle_right;

```

```

6     float_t angle_camera;
7 } angles; // 当前角度（度）
8
9 struct {
10     uint32_t chassis_lock : 1;
11     uint32_t motor0_break : 1;
12     uint32_t motor1_break : 1;
13     uint32_t diff_locked : 1;
14 #ifdef UART_PID_DEBUG
15     uint32_t pid_valid : 1; // 可忽略
16 #endif
17 } flags;
18
19 struct {
20     int8_t left;
21     int8_t right;
22     uint16_t reserved; // 保留
23     uint32_t remain_pulses; // 剩余脉冲数（编码器计数）
24 #ifdef UART_PID_DEBUG
25     float kp, ki, kd; // 可忽略
26 #endif
27     float left_laps; // 左轮累计圈数
28     float right_laps; // 右轮累计圈数
29 } motor_rpm;
30
31 struct {
32     uint32_t red_mode : 2;
33     uint32_t blue_mode : 2;
34     uint32_t defenses_mode : 2;
35     uint32_t flow_enable : 1;
36     uint32_t period_ms : 12;
37     uint32_t duty_cycle : 7;
38     uint32_t reserved : 6;
39 } led_ctrl;
40
41     uint8_t result; // 执行结果：0=成功，1=失败
42 } __attribute__((packed)) status_feedback_t;

```

关键字段说明：

- `angles.*`：当前姿态角（度），最近一次命令值。
- `flags.*`：当前锁、刹车、差速状态。
- `motor_rpm.left/right`：当前电机占空（实时值）；
- `motor_rpm.remain_pulses`：剩余脉冲数（编码器计数），用于判断剩余行程。
- `motor_rpm.left_laps/right_laps`：左右轮累计圈数。
- `led_ctrl.*`：当前指示灯配置。
- `result`：最近一次指令的执行结果（0=成功，1=失败）。

交互约定：

- 查询：第三方发送类型为 `STATUS_FEEDBACK` 的请求帧；AGM 忽略请求体内容，返回同类型的完整 `status_feedback_t`。

5. 典型交互与时序

- SENSOR_DATA: AGM 以 10Hz 主动发送 `sensor_data_t`，用于第三方侧感知定位/IMU/超声。
- CONTROL_CMD: 第三方发送 `ctrl_frame_t` 触发运动/灯控等；AGM 按命令执行。
- STATUS_FEEDBACK: 第三方按需发送查询；AGM 忽略请求体并返回完整状态。

6. API 使用范式（最小示例）

最小控制发送:

```
1  #include "uart_frame.h"
2
3  void send_ctrl_example(void) {
4      ctrl_frame_t ctrl = {0};
5      ctrl.angles.angle_left = 15.0f;
6      ctrl.angles.angle_right = -10.0f;
7      ctrl.angles.angle_camera = 5.0f;
8      ctrl.flags.chassis_lock = 0;
9      ctrl.motor_rpm.left = 30;    // [-100,100]
10     ctrl.motor_rpm.right = 30;   // [-100,100]
11     ctrl.motor_rpm.pulses = 186; // 1 圈 = 186 脉冲
12
13     uint8_t tx_buf[256];
14     uint16_t frame_len;
15     int32_t rc = uart_frame_pack(UART_FRAME_TYPE_CONTROL_CMD,
16                                 &ctrl, sizeof(ctrl),
17                                 tx_buf, sizeof(tx_buf), &frame_len);
18     if (rc != UART_FRAME_OK)
19     {
20         rt_kprintf("[ERROR] Pack failed: %s\n",
21                     uart_frame_get_error_string(rc));
22     }
23     else
24     {
25         uart_send(tx_buf, frame_len);
26     }
27 }
```

最小解析:

```
1  #include "uart_frame.h"
2
3  int8_t parse_rx(const uint8_t* rx, size_t len) {
4      uart_frame_parse_result_t r;
5      int32_t ret = uart_frame_parse(rx, len, &r);
6      if (ret != UART_FRAME_OK)
7      {
8          printf("failed parse rv1106 buf: %s\n",
9                  uart_frame_get_error_string(ret));
10         return -1;
11     }
12     switch(r.type)
```

```

13     {
14         case UART_FRAME_TYPE_CONTROL_CMD:
15             //todo:
16             break;
17         case UART_FRAME_TYPE_STATUS_FEEDBACK:
18             //todo:
19             break;
20         case UART_FRAME_TYPE_SENSOR_DATA:
21             //todo:
22             break;
23         default: printf("unknown uart_frame_type\n"); break;
24     }
25     return 0;
26 }

```

查询 STATUS_FEEDBACK (请求体全零) :

```

1  #include "uart_frame.h"
2
3  void query_status(void) {
4      status_feedback_t req = {0}; // AGM 忽略请求体
5
6      uint8_t tx_buf2[128];
7      uint16_t frame_len2;
8      int32_t rc = uart_frame_pack(UART_FRAME_TYPE_STATUS_FEEDBACK,
9                                   &req, sizeof(req),
10                                  tx_buf2, sizeof(tx_buf2), &frame_len2);
11      if (rc == UART_FRAME_OK) {
12          uart_send(tx_buf2, frame_len2);
13      }
14  }

```

7. 误用与排障

- 头尾校验失败：检查魔术字是否因粘包/截断受影响，可用 `uart_frame_find()` 从流中定位帧边界。
- 对齐与端序：确保严格使用文中结构体定义 (packed) 与小端序。
- 浮点跨平台：若需跨语言解析，建议仅依赖整数域；浮点域用于观测与调试。

8. 版本与兼容

- 字段演进：新增字段仅在结构体尾部追加；旧字段语义保持不变。
- 版本管理：当存在破坏性变更时提升协议版本；双方以 `version` 字段进行兼容校验。